

main.py

Visualize

Run

```
1 from functools import lru_cache
2
3 def all_segmentations(s, wordDict):
4     wordSet = set(wordDict)
5
6     @lru_cache(None)
7     def dfs(start):
8         if start == len(s):
9             return [""]
10
11         result = []
12
13         for end in range(start+1, len(s)+1):
14             word = s[start:end]
15
16             if word in wordSet:
17                 for sub in dfs(end):
18                     if sub:
19                         result.append(word + " " + sub)
20                     else:
21                         result.append(word)
22
23         return result
24
25     return dfs(0)
26
27 print(all_segmentations(
```

Output

['cat sand dog', 'cats and dog']

=== Code Execution Successful ===

main.py

Visualize

Run

Output



```
1 def greedy_wrap_cost(words, M):
2
3     total = 0
4     i = 0
5     n = len(words)
6
7     while i < n:
8
9         line_len = words[i]
10        j = i + 1
11
12        while j < n and line_len + 1 + words[j] <= M:
13            line_len += 1 + words[j]
14            j += 1
15
16        if j != n:
17            total += (M - line_len) ** 2
18
19        i = j
20
21    return total
22
23 print(greedy_wrap_cost([3,2,2,5],6))
```

16

=== Code Execution Successful ===

main.py

Visualize

Run

Output

```
1 def wrap_lines(words, M):
2
3     n = len(words)
4
5     dp = [0]*(n+1)
6     parent = [0]*n
7
8     for i in range(n-1,-1,-1):
9
10        dp[i] = float('inf')
11        line_len = -1
12
13        for j in range(i,n):
14
15            line_len += words[j] + 1
16
17            if line_len > M:
18                break
19
20            if j == n-1:
21                cost = 0
22            else:
23                cost = (M-line_len)**2
24
25            if cost + dp[j+1] < dp[i]:
26                dp[i] = cost + dp[j+1]
27                parent[i] = j+1
```

[[3], [2, 2], [5]]

=== Code Execution Successful ===

main.py

Visualize

Run

```
1 def min_wrap_cost(words, M):
2
3     n = len(words)
4
5     dp = [0]*(n+1)
6
7     for i in range(n-1, -1, -1):
8
9         dp[i] = float('inf')
10        line_len = -1
11
12        for j in range(i, n):
13
14            line_len += words[j] + 1
15
16            if line_len > M:
17                break
18
19            if j == n-1:
20                cost = 0
21            else:
22                cost = (M-line_len)**2
23
24            dp[i] = min(dp[i],
25                      cost + dp[j+1])
26
27    return dp[0]
```

Output

10

=== Code Execution Successful ===